VNAV Lab4 无人机控制与竞速

黄博宇 24020036026；中外计科一班；第8组

实验记录：2026年10月22日；报告修订：2026年10月23日

摘要

这次实验把Lab3控制器接到Lab4轨迹程序上，先测试第一门悬停，再完成一次全赛道飞行。程序从实际位置出发，经过12个赛门，以50Hz发送目标位置、速度、加速度和朝向。对保存的bag进行时间对齐后，执行段三维位置RMSE为5.64cm，最大误差为8.44cm。主要偏差在高度：实际位置平均比目标低5.47cm。本轮验证了整套程序可以协同运行；高度偏差原因和提速效果尚未做对照验证。

1 引言

Lab3解决的是“怎样跟上目标”，Lab4进一步解决“每个时刻应该飞到哪里”。首门悬停只需要一个固定目标。连续穿门则需要一条带时间的路线，不能只把下一道门的位置交给控制器。

我先用第一门检查位置、通信和基本控制，再运行完整路线。轨迹程序像路线规划员，安排位置与朝向；控制器像驾驶员，根据实际状态纠偏。这就是闭环：不断看偏差，再调整旋翼。[1,2]

本次补全Part0与Part1.1至Part1.3，保存视频、截图和消息记录，并完成两道个人理论题。运行目标是稳定通关，没有进行多轮最快用时比较。

2 实验设置

表1列出本轮实际使用的配置。所有统计都对应这一配置，没有混入其他参数的结果。

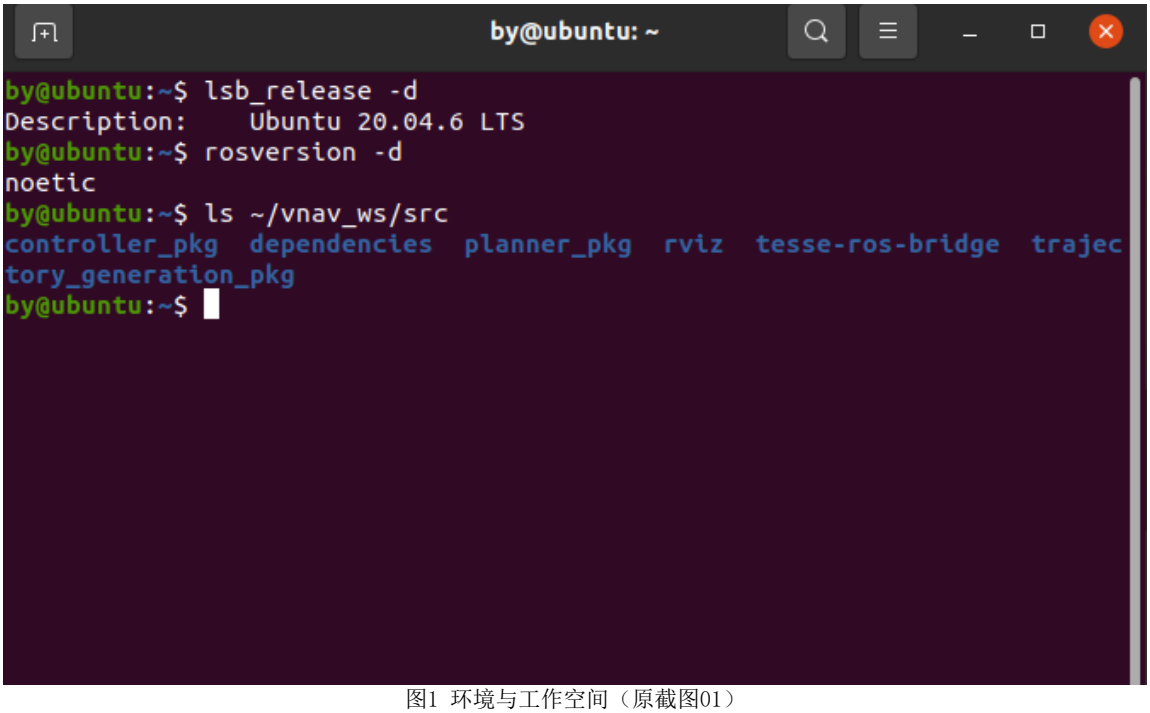


图1 环境与工作空间（原截图01）

表1 实验环境与运行参数

项目	本次设置	作用
系统与框架	Ubuntu 20.04.6、ROS Noetic	运行与连接ROS程序
仿真与代码	TESSE Unity Lab4、VNAV2021	提供无人机、赛道和实验框架
控制器增益	kx=2、kv=3、kr=3、komega=1	调节位置、速度、姿态和角速度的反馈强度
时间倍率	time_scale=2.0	将估计的每段时间乘2
目标发布频率	50Hz	约每0.02s更新一次目标
时间估计参数	v_max=8.0、a_max=4.0	用于估计段时间，并非严格飞行上限
位置与朝向维度	3与1	分别规划XYZ和yaw
多项式系数数目	N=10	每段最高九次多项式

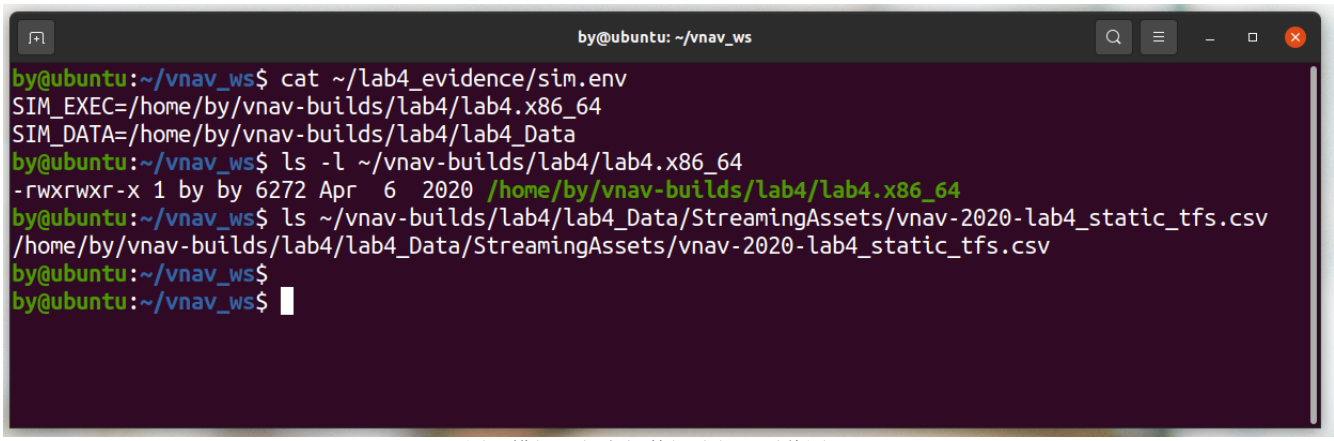


图2 模拟器与赛门数据路径（原截图03）

ROS是供不同程序交换消息的框架；其中一个程序称为节点，消息传输通道称为话题。通信桥将仿真中的实际状态送到/current_state。赛门发布器读取赛门文件，轨迹节点接收当前位置与赛门目标，再向/desired_state发布期望状态。控制器输出/rotor_speed_cmds，驱动仿真无人机。

2.1 轨迹与控制实现

代码按四个Part补全，具体内容见表2。核心实现位于simple_traj_planner.cpp和trajectory_generation_node.cpp。

表2 各部分实现及设计理由

部分	实现内容	设计理由
Part0	第一门设为固定目标，期望速度和加速度设为0	在连续飞行前确认坐标与控制链路
Part1.1	读取当前位置和yaw，确认数值有效	避免从未初始化的状态规划
Part1.2	添加起点与赛门约束，分别优化位置和yaw	满足目标并协调转向
Part1.3	按经过时间采样，发布位置、速度、加速度和朝向	将整条轨迹转为控制器可执行的当前目标

Yaw是从上方看机头的朝向角，相当于无人机的“指南针方向”。它跨过±180°时会发生数值跳变，例如179°到-179°只需转2°。代码用atan2(sin Δψ, cos Δψ)选择相邻目标间的短角度变化，再累加为连续朝向，避免把2°误认为358°。

位置轨迹有三个分量，朝向只有一个分量，二者分别求解，但共用分段时间。因此到达某道门时，位置和机头朝向对应同一时刻。起点使用实际状态，起终点通过makeStartOrEnd设置边界条件；内部航点固定目标位置与yaw。滚转、俯仰由控制器根据运动需求调整，不单独指定完整路线。

Minimum snap指最小化位置四阶导数的平方积分，以减少参考轨迹过于猛烈的变化。代码先估计各段时间，再乘时间倍率，随后在线性优化器中求出多项式系数。这里优化的是参考轨迹，不直接保证旋翼能耗最小，也不包含障碍物在线重规划。[3]

每次发布时，采样时间取“当前ROS时间减轨迹开始时间”，并限制在0到总时长之间。目标位姿写入transforms，线速度和线加速度分别写入velocities、accelerations。Yaw转换为四元数，即用四个数表示三维方向。到达结束时刻后保持末端目标。本次沿用Lab3的简化控制器，没有加入期望角速度前馈，即没有提前补偿目标转向速度。

2.2 实验步骤

操作顺序采用“先检查基本控制，再记录完整运行”。

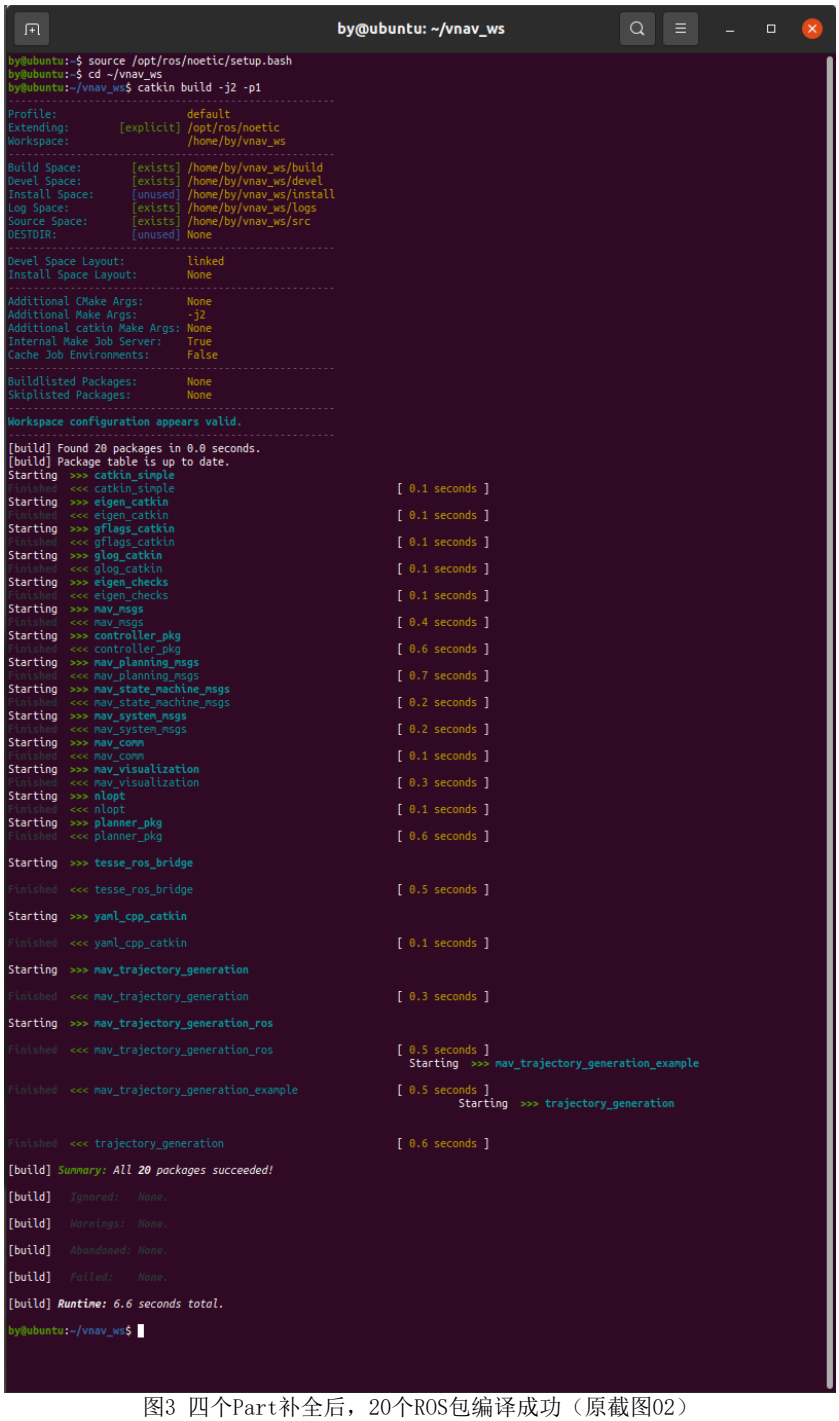


图3 四个Part补全后，20个ROS包编译成功（原截图02）

表3 实验步骤与检查结果

步骤	操作与目的	检查结果
1 环境准备	补齐依赖后编译工作空间	20个包全部编译成功
2 首门测试	启动仿真、通信桥与固定目标	无人机在第一门附近稳定保持
3 切换模式	停止固定目标，复位并重启跟随节点	避免两套程序同时发送目标
4 录制飞行	先启动跟随器和记录器，最后发布赛门	保存完整视频及bag
5 核验材料	播放视频并检查状态话题	视频正常，期望与实际状态均有记录

仿真复位不会清除轨迹节点中的旧路线，因此重跑时还需要重启跟随节点。赛门消息重复发布也不会触发已有轨迹从头生成，这能避免正常飞行中路线被反复重置。

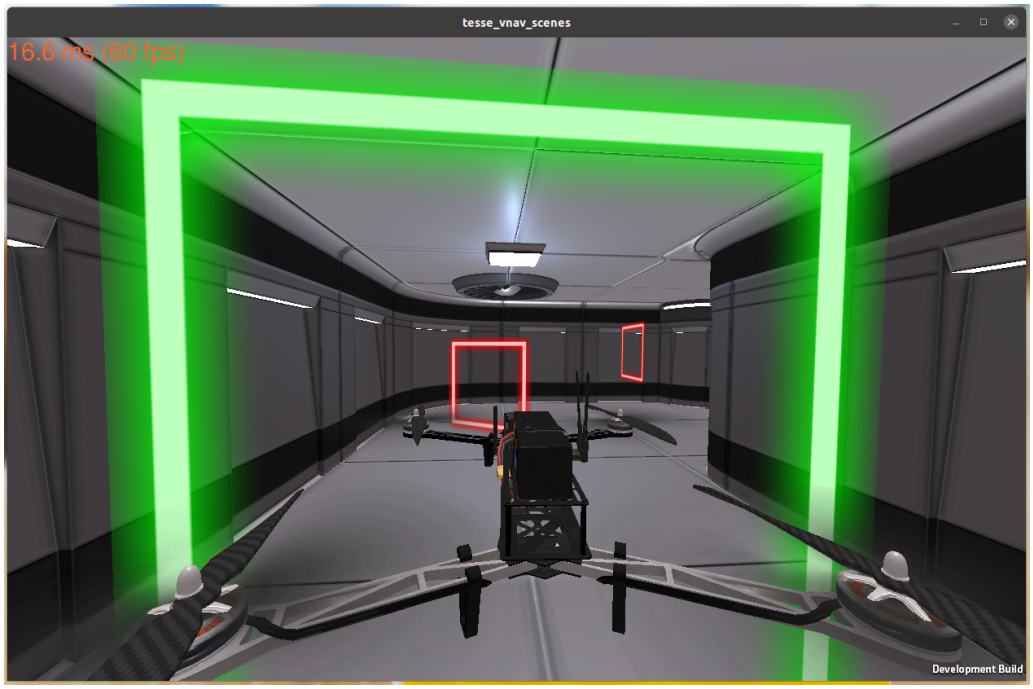


图4 第一门附近稳定悬停，先确认基本控制正常（原截图04）

2.3 数据记录与处理

用于分析的是run_20261022_000059。Bag是按时间保存ROS消息的文件。本记录包含40534条消息，其中实际状态8365条、期望状态7836条，还保留了仿真时钟、赛门目标与旋翼命令。

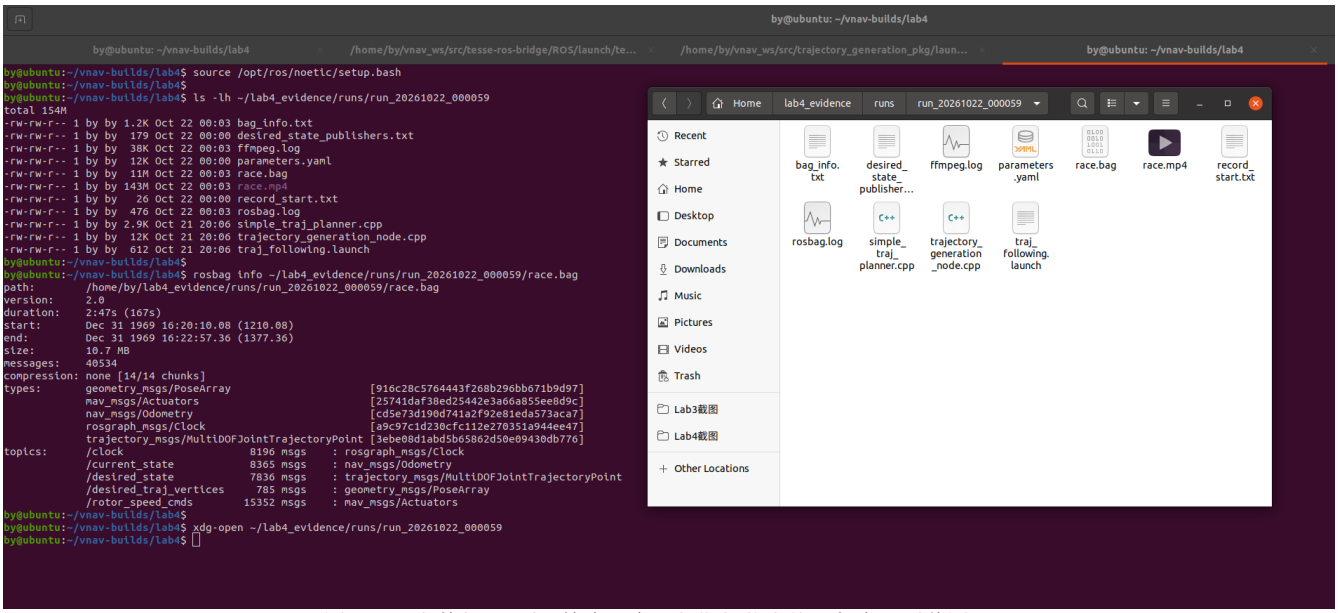


图5 记录文件与bag话题检查；实际与期望状态均已保存（原截图08）

两类状态的消息数量不同，不能按数组下标直接相减。本次使用bag记录时间，将期望位置线性插值到实际状态时刻；也就是用相邻两个目标值估计中间时刻的目标。Yaw先展开再插值。利用持续推进阶段的“记录时间减轨迹经过时间”估计轨迹起点，并排除结束后的保持阶段，得到6708个执行段样本。

位置误差定义为实际位置减期望位置。三维RMSE（均方根误差）为$\mathrm{sqrt}(\mathrm{mean}(ex^2+ey^2+ez^2))$，它综合各采样点偏差，并对较大的误差赋予更高权重。各轴平均有符号误差用来区分持续偏移与正负波动。[4]

2.4 为什么这样设计

轨迹生成前先拿到实际起点，是为了避免无人机从默认坐标突然跳到一条不属于当前位置的路线。固定目标测试通过后，才把目标换成随时间推进的轨迹。表4把几个容易混淆的选择放在一起。

表4 关键设计与原因

选择	为什么这样做	本次边界
实际位置与yaw作起点	起点和当前状态一致，减少参考突变	仍需等待有效状态；不是任意状态下都能立即起飞
位置与yaw分别优化、共用时间	到门时，位置与朝向目标同步	不单独规划完整滚转和俯仰曲线
时间倍率2.0	给跟踪留出更多执行时间	未做其他倍率的实测对比
只在首次有效航点到达后求解	重复收到赛门时不重置路线	重跑必须重启跟随节点
末端时间截断	不在轨迹范围外取值，到终点后保持	轨迹结束不等于关闭控制器

对同一条曲线，如果只是把执行时间拉长两倍，速度会减为一半，加速度会减为四分之一。这能解释为什么时间分配会影响跟踪难度。但本次没有另一倍率的对照运行，不能把这个时间缩放关系写成实测提升。

2.5 如何确认数据链路完整

模拟器有画面，只说明赛道打开了，还要检查消息是否传通。实际状态、目标状态和旋翼命令分别负责反馈、参考和执行，三者缺一都会影响飞行。

表5 主要消息及检查意义

话题	本记录条数	负责什么
/current_state	8365	返回实际位置、方向和速度
/desired_state	7836	发布此刻应该到达的目标状态
/desired_traj_vertices	785	提供按顺序排列的赛门目标
/rotor_speed_cmds	15352	将控制器计算结果送给旋翼接口
/clock	8196	提供仿真时间，支持按时间推进

记录条数不同并不直接说明丢包，因为各话题的发布频率和有效时间范围不同。对误差分析而言，关键是期望与实际状态有共同覆盖区间，并按同一时间基准比较，而不是让两个数组长度相同。

3 实验结果

下面先列测量结果，再讨论它们能说明什么。悬停读数和竞速统计分别报告。

3.1 首门悬停

截图时刻的目标与实际位置见表6。水平方向接近重合，实际高度比目标低约3.96cm。这是一次读数，不代表竞速全程平均值。

表6 首门悬停的一次位置读数，单位m

方向	期望位置	实际位置	实际减期望
X	0.4226201	0.4226182	-0.0000019
Y	3.1652320	3.1652690	+0.0000370
Z	2.6700000	2.6304210	-0.0395790

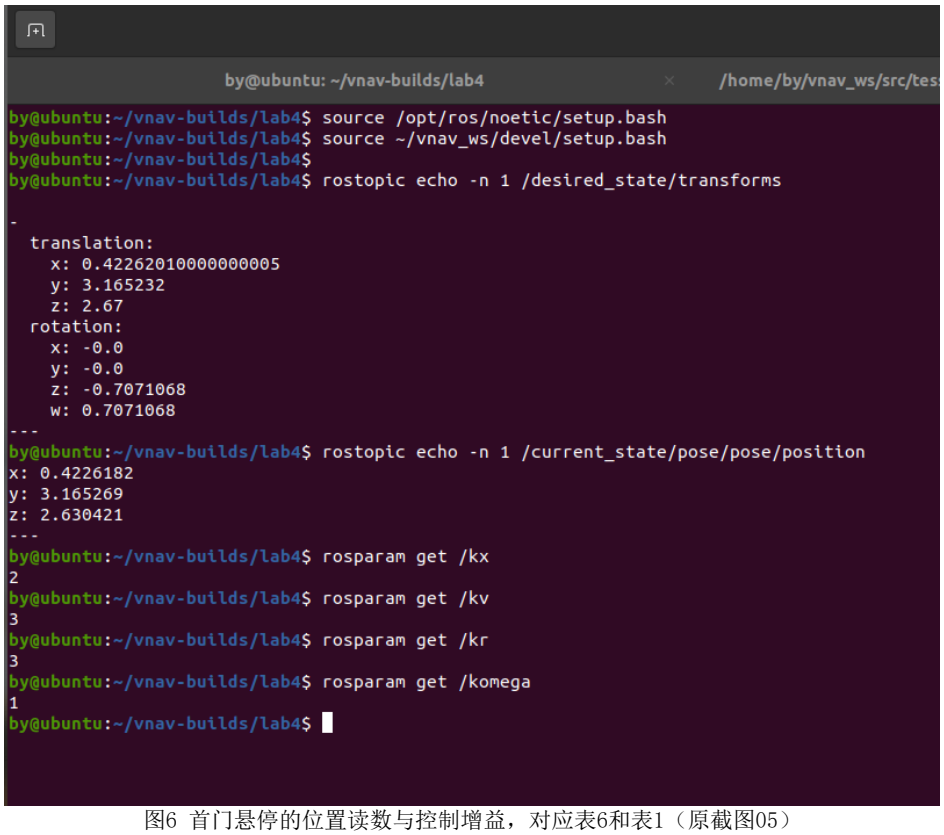


图6 首门悬停的位置读数与控制增益，对应表6和表1（原截图05）

3.2 完整赛道运行

保存的连续视频记录了一次完整赛道运行，最后一道门后的画面见图8。统计包含12个赛门目标，不含额外加入的当前起点。

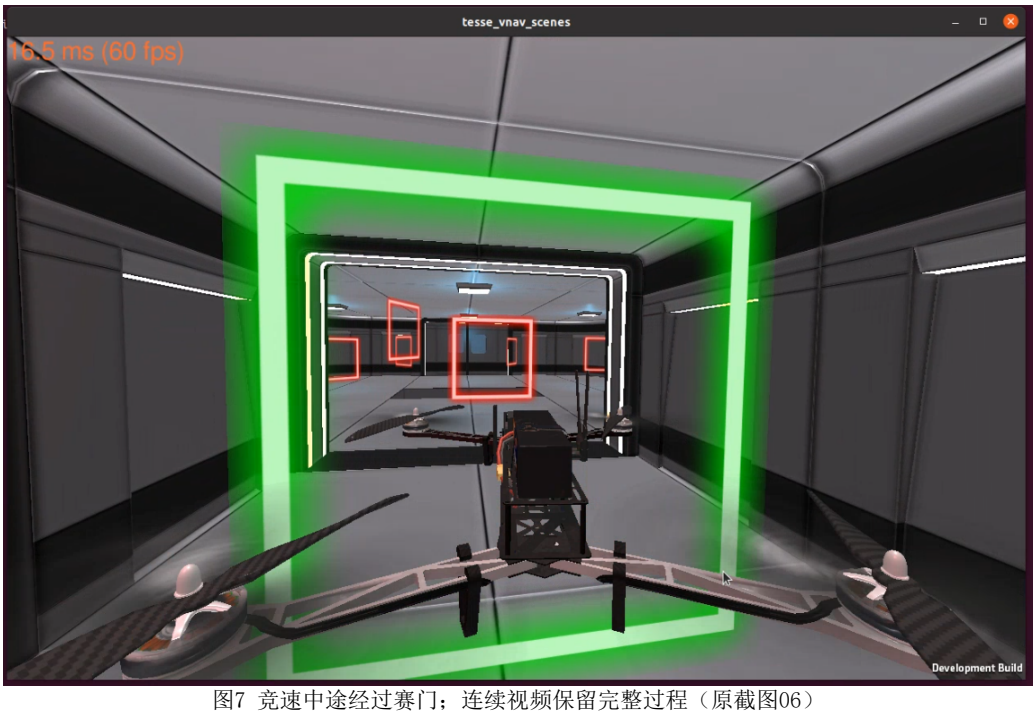


图7 竞速中途经过赛门；连续视频保留完整过程（原截图06）

表7 轨迹执行段的主要结果

指标	结果
有效样本	6708
三维位置RMSE	0.056426m
平均 / 中位位置误差	0.055555 / 0.056864m
95%分位 / 最大位置误差	0.068054 / 0.084428m
X / Y / Z方向RMSE	0.007171 / 0.006392 / 0.055602m
X / Y / Z平均有符号误差	-0.000647 / +0.000435 / -0.054671m
Yaw RMSE	2.2922°
最高 / 平均实际速度	1.6292 / 0.7125m/s
采样累计实际路径长度	95.570m

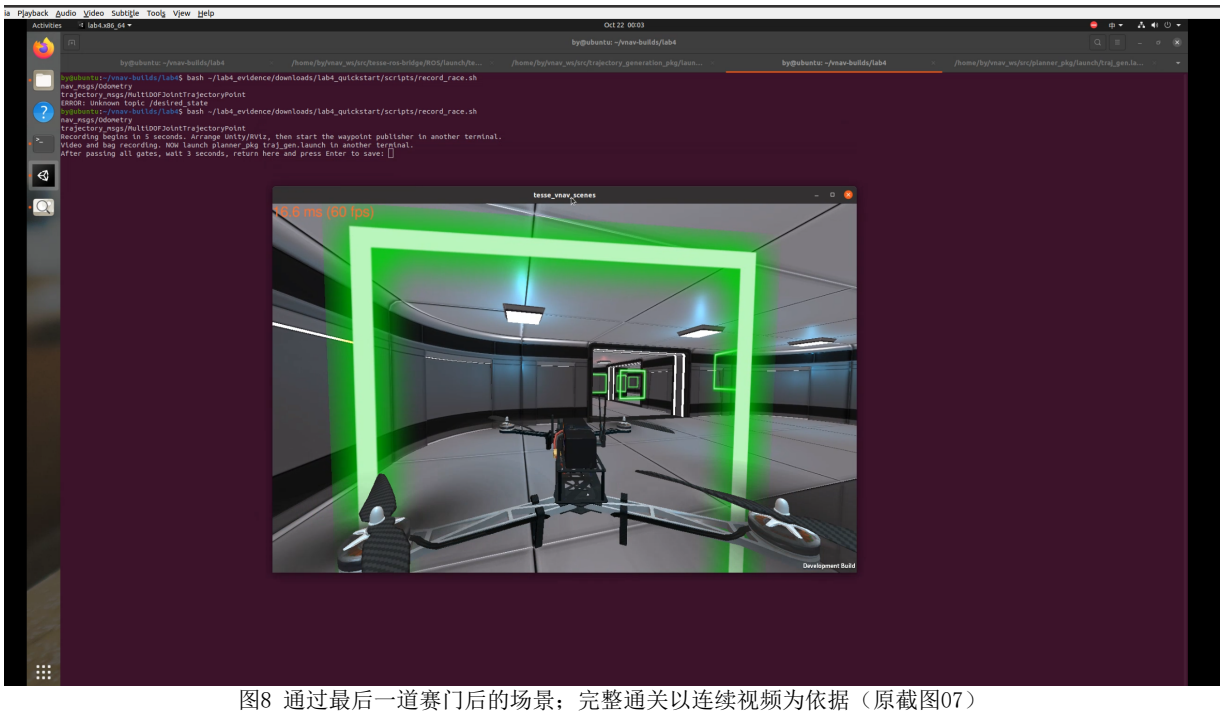


图8 通过最后一道赛门后的场景：完整通关以连续视频为依据（原截图07）

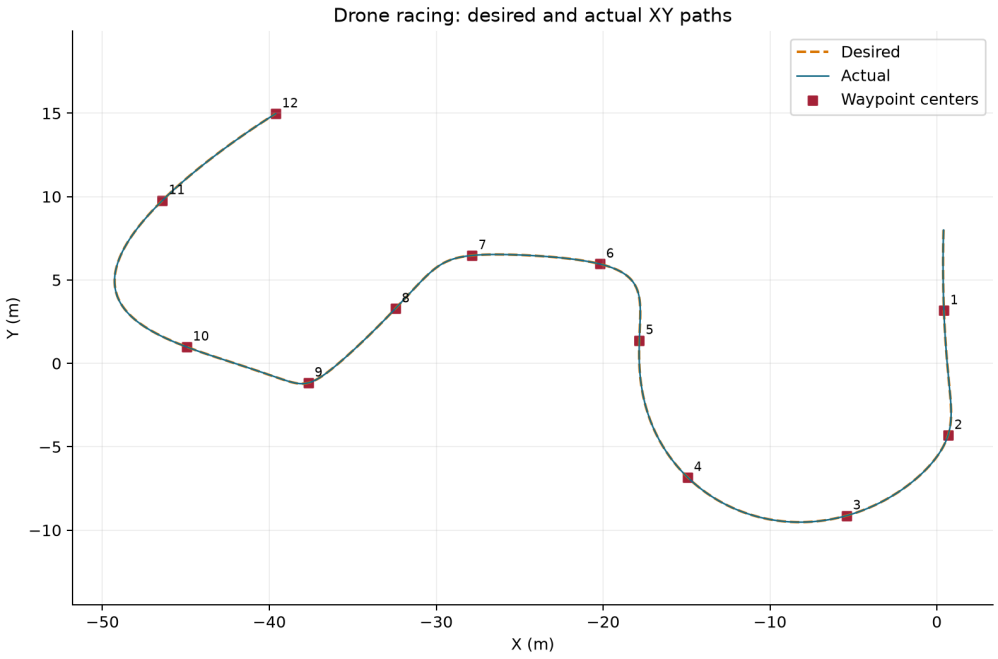


图9 水平轨迹对比：期望路线与实际路线

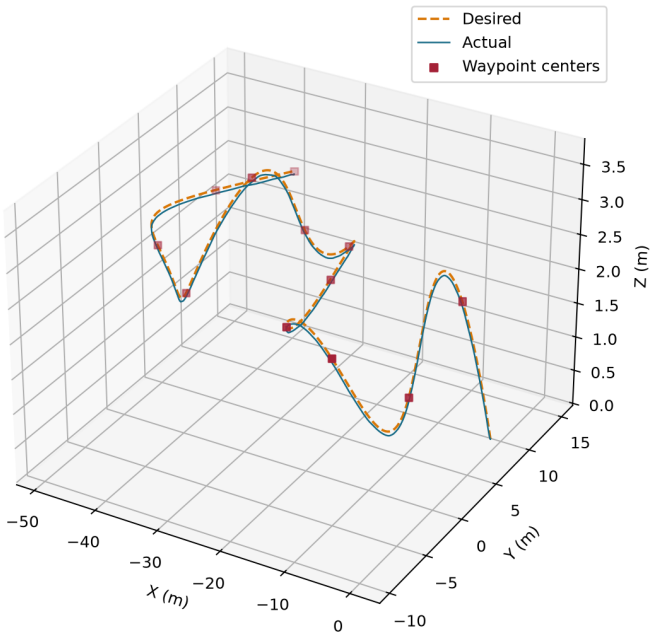


图10 三维轨迹与赛门中心：同时查看位置和高度变化

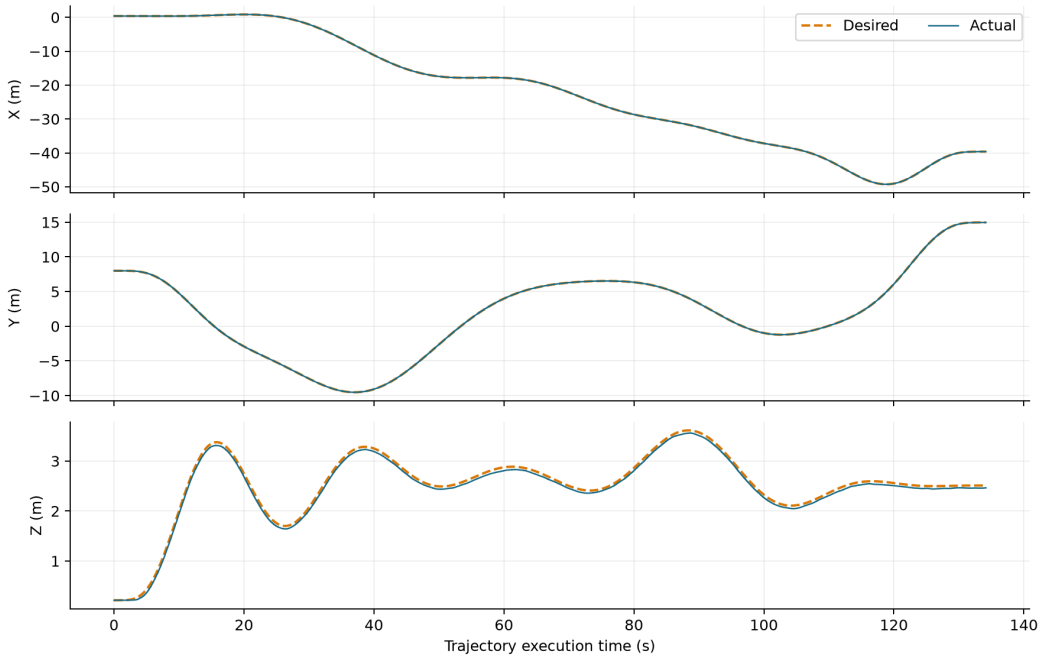


图11 XYZ位置随时间变化：查看各轴的跟踪情况

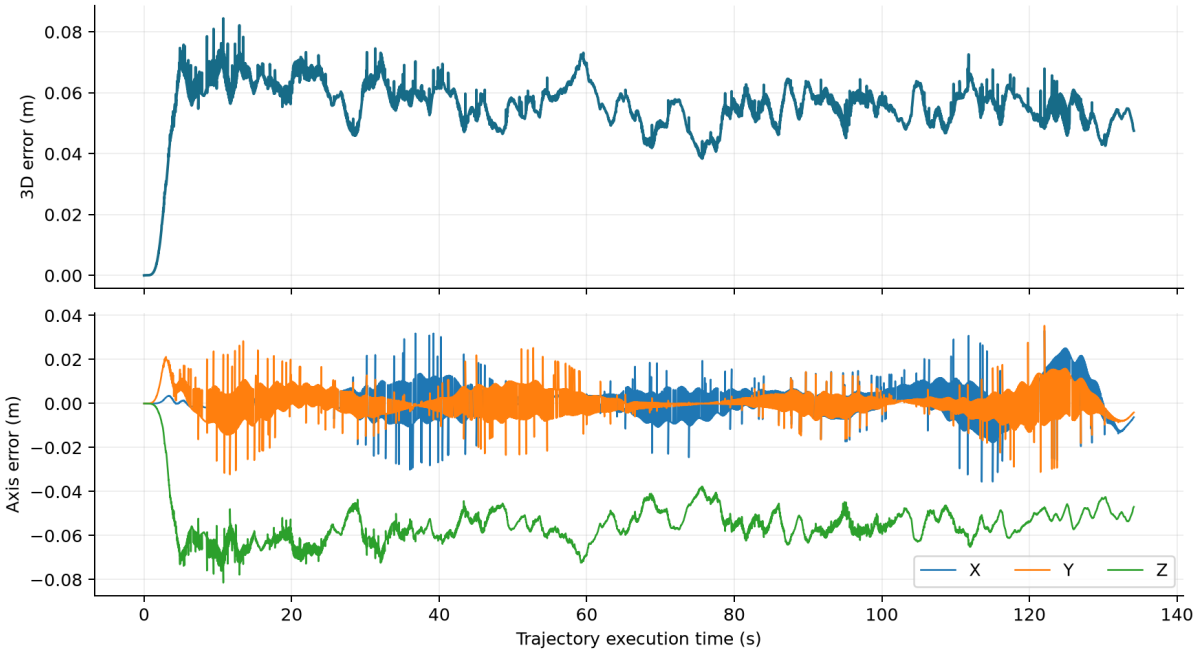


图12 位置误差随时间变化：高度偏差最明显



图13 各轴位置RMSE对比：高度误差大于水平误差

3.3 时间统计

三个文件或程序的起止范围不同，因此它们的时长不应互相代替。

表8 记录时长与参考轨迹时长

项目	时长	含义
Bag记录	167.28s	包含准备、执行和末端保持
录屏视频	160.75s	实际录屏文件播放长度
参考轨迹	134.17s	程序安排的轨迹执行时间
独立裁判竞速计时	[待补充]	未提供独立计时记录

4 讨论

这轮能飞完整条路线，但“跟得准”“为什么有误差”和“跑得快”是不同问题。下面分别讨论。

4.1 跟踪效果与高度偏差

水平方向RMSE分别约0.72cm和0.64cm，明显小于Z方向的5.56cm。Z方向平均误差为-5.47cm，与该方向RMSE接近，说明高度误差含有持续向下的偏移，而不仅是正负随机波动。首门悬停也出现高度偏低，但单点读数与完整执行段属于不同统计口径，不能用两者的差值推断偏差随时间增长。

本记录中95%分位误差约6.81cm，最大约8.44cm，可与平均误差一起描述这一轮的跟踪分布。它们不能用于推断其他参数、其他路线或真实无人机的表现。现有数据没有对控制参数和模型项逐一做隔离实验，因此无法确定高度偏差的具体原因。

4.2 轨迹优化与理论结果的关系

单段理论题把时间与位移固定，再最小化速度平方积分。平方会放大速度峰值，因此原始位置约束下匀速最优；这只是题目选定的代价，不等于无人机耗电。当只有P(0)=0、P(1)=1时，最优P(t)=t，增加到二次或三次不会降低代价J=1。加上P’(1)=-2后，二次解为$4t-3t^2$，J=4；三次解为$t/4+9t^2/2-15t^3/4$，J=17/8。这个对比说明，多项式次数只有在新增自由度能满足约束并改善目标时才有作用。

多段最小snap理论在一维、分段时间固定、航点位置已知且起终点低阶导数固定的条件下，每段可用七次多项式表示。k段共有8k个系数：位置条件2k个，起终点导数条件6个，内部低阶连续条件3(k-1)个，自然最优条件3(k-1)个。自然条件来自求最优解，不能全部当成外加约束。例如三段共有24个系数，显式约束18个，自然条件6个。

代码使用N=10，即每段10个系数、最高九次；它与上述七次理论的边界设置和实现选择不同，不能把N直接改成7。两份个人理论题已分别整理为英文deliverable_1.pdf和deliverable_2.pdf，保留矩阵、求解和代回检查。

4.3 数据与实验方法的局限

误差相对于仿真发布的实际状态计算，并非外部测量系统验证的定位误差。记录时间对齐不能完全分离通信延迟与控制误差。全重叠时段RMSE为0.054428m，因包含末端保持而与执行段不同，本报告主要采用后者。记录中未发现时钟回退，但这并不消除其他采样与通信影响。

静态截图只能说明某一时刻的状态；到赛门中心的最近距离也不能独立证明正确穿门。完整通关判断依靠连续视频和本人运行检查，数值分析用于补充跟踪表现。采样点累计路径长度同样不是精确赛道几何长度。

本次只有一轮完整材料，也没有独立裁判计时，因此134.17s仅称为参考轨迹时长。时间倍率2.0给跟踪留下了更多时间，但没有倍率1.0或其他设置的对照，不能量化其改善幅度。

4.4 调试经验与后续工作

环境准备时，gflags_catkin缺失通过补齐源码解决；catkin_simple指向旧目录的问题通过备份旧build/devel后重建解决；NLopt下载连接被拒绝后，改用校验过的本地安装包，最终20个包全部编译成功。第一次录制无法找到/desired_state，确认话题就绪后第二次正常保存。这些问题分别属于依赖、缓存、下载与启动时序，需要按错误所在环节处理。

后续可先固定路线与时间倍率，每次只改变一个控制或模型参数，比较高度平均偏差、最大误差和通关情况。稳定后再逐步缩短分段时间，并用多轮结果比较成功率和用时。本报告没有将这些计划写成已完成的实验。

4.5 把高度误差再拆开看

只看三维RMSE，很难判断是水平偏差还是高度偏差。用各轴RMSE可进一步计算水平合成误差；再把Z方向均方误差拆成平均偏移平方与去均值波动两部分。

表9 从原统计量得到的补充诊断

诊断量	结果	计算与含义
水平合成RMSE	0.96cm	sqrt(RMSE_X^2+RMSE_Y^2)，合并X与Y
高度占三维均方误差的比例	97.10%	RMSE_Z^2 / RMSE_3D^2，说明高度是主要误差方向
平均偏移平方占Z均方误差比例	96.68%	mean(eZ)^2 / mean(eZ^2)，说明持续偏移占主导
高度去均值后的均方根波动	1.01cm	sqrt(mean(eZ^2)-mean(eZ)^2)，描述围绕平均偏差的变化

这些数值只是对同一轮记录的重新分解，不是新增实验。均方误差按各轴相加，RMSE本身不能直接相加。偏移占比高，也不意味着可以直接把目标高度抬高相同数值作为通用修复；那样可能只遮住现象，无法验证原因。

4.6 怎样安排下一轮验证

后续应把“检查原因”和“争取更快”分开。先保证同一设置能重复完成，再改变一个因素，避免同时改多个参数后无法判断是哪一项起作用。

表10 后续验证计划，尚未执行

顺序	只改变或检查什么	比较什么
1 重复基准运行	保持路线、增益与倍率不变	通关情况、平均高度偏差、最大误差
2 检查高度问题	核对坐标、重力补偿和模型约定；每次只验证一项	高度偏差是否稳定改变
3 调整单个控制增益	其余参数固定	偏差、波动和通关情况是否同时改善
4 逐步缩短分段时间	在稳定设置上减小时间倍率	多轮成功率与独立记录的完成时间

5 结论

四个Part已经接通：固定目标悬停、读取实际起点、规划位置和朝向、按时间发送目标。保存的一轮运行完成了赛道，执行段三维位置RMSE为5.64cm，最大误差为8.44cm。水平误差小，高度持续偏低；现有记录还不能确定原因。

下一步先固定路线和时间倍率，逐项检查高度偏差，再用多轮运行比较提速后的效果。现有代码、原视频、bag和截图均已保存。两份个人理论题使用英文交付；指定Drive/Dropbox媒体公开链接仍为[待补充]。

参考文献

[1] MIT VNAV. Lab 4: UAV Control and Racing.
https://vnav.mit.edu/labs_2023/lab4/exercises.html

[2] MIT-SPARK. VNAV-labs, VNAV2021, lab4.
https://github.com/MIT-SPARK/VNAV-labs/tree/VNAV2021/lab4

[3] ETH Zurich ASL. mav_trajectory_generation.
https://github.com/ethz-asl/mav_trajectory_generation

[4] 本实验记录run_20261022_000059及仓库data/analysis/metrics.json、analyze_run.py；理论题见deliverable_1.pdf与deliverable_2.pdf。

实验运行、截图和视频来自本人Ubuntu环境。

附录A 任务要求的核心代码与注释

以下摘录来自归档运行代码，保留计算与消息赋值，补充解释性注释；为阅读省去模板说明、重复空行和进度日志。A.3的有效状态检查省略告警文本，其他设置与实际实现一致。摘录不是独立可编译文件，完整上下文见code目录。

表A1 任务、函数与验证材料

任务	核心函数	验证材料
Part0	trajCB	首门悬停画面、目标与实际位置
Part1.1	onCurrentState	当前状态话题、起点有效性检查
Part1.2	generateOptimizedTrajectory	航点与轨迹图、连续朝向处理
Part1.3	publishDesiredState	期望状态话题、执行段误差图

A.1 Part0

```
// 取第一道赛门；上游已检查数组非空。
const auto& pose = traj_msg.poses.front();
trajectory_msgs::MultiDOFJointTrajectoryPoint point;
geometry_msgs::Transform transform;
transform.translation.x = pose.position.x;
transform.translation.y = pose.position.y;
transform.translation.z = pose.position.z;
transform.rotation = pose.orientation;
// 位姿必须放入消息数组，不能只创建局部变量。
point.transforms.push_back(transform);
// 默认构造的Twist线速度与角速度均为0，表示悬停。
point.velocities.emplace_back();
// 同样加入零加速度项，保证消息字段齐全。
point.accelerations.emplace_back();
point.time_from_start = ros::Duration(0.0);
// 发送固定目标，控制器据此计算修正量。
desired_state_pub.publish(point);
// 以下构建world到desired_pose的坐标变换，便于查看目标。
tf::Transform desired;
desired.setOrigin(tf::Vector3(pose.position.x, pose.position.y, pose.position.z));
tf::Quaternion q;
tf::quaternionMsgToTF(pose.orientation, q);
// 归一化方向四元数，再广播目标坐标关系。
q.normalize();
desired.setRotation(q);
br->sendTransform(tf::StampedTransform(desired, ros::Time::now(), "world", "desired_pose"));
```

A.2 Part1.1

```
// 从里程计消息读取世界坐标中的XYZ，作为轨迹起点。
x << cur_state.pose.pose.position.x, cur_state.pose.pose.position.y,
     cur_state.pose.pose.position.z;
// 从当前四元数提取水平朝向。
current_yaw = tf::getYaw(cur_state.pose.pose.orientation);
// 仅接受有限数值，避免NaN或无穷大进入优化器。
have_current = x.allFinite() && std::isfinite(current_yaw);
```

A.3 Part1.2

```
// 有效状态未到达前不规划；此处省略原代码的告警文本。
if (!have_current) return;
// 已有路线则直接返回，避免重复赛门消息重置计时。
if (!trajectory.empty()) return;
// 位置需要XYZ三个维度；yaw单独用一维优化。
const int D = 3;  // dimension of each vertex in the trajectory
mav_trajectory_generation::Vertex start_position(D), end_position(D);
mav_trajectory_generation::Vertex::Vector vertices;
mav_trajectory_generation::Vertex start_yaw(1), end_yaw(1);
mav_trajectory_generation::Vertex::Vector yaw_vertices;
using namespace mav_trajectory_generation::derivative_order;
// 固定实际起点，并通过库接口设置起点导数边界。
start_position.makeStartOrEnd(x, SNAP);
vertices.push_back(start_position);
Eigen::VectorXd initial_yaw(1); initial_yaw << current_yaw;
// 朝向从实际yaw开始，不能无条件设为0。
start_yaw.makeStartOrEnd(initial_yaw, SNAP);
yaw_vertices.push_back(start_yaw);
double last_yaw = current_yaw;
// 逐门添加位置和朝向；同一索引对应同一门。
for (auto i = 0; i < poseArray.poses.size(); ++i) {
// 读取当前索引对应的赛门位姿。
  const auto& pose = poseArray.poses[i];
  Eigen::Vector3d position(pose.position.x, pose.position.y, pose.position.z);
  mav_trajectory_generation::Vertex vertex(D);
// 最后一道门使用终点边界；中间门仅固定目标值。
  if (i + 1 == poseArray.poses.size())
    vertex.makeStartOrEnd(position, SNAP);
  else
    vertex.addConstraint(POSITION, position);
  vertices.push_back(vertex);
  double yaw = tf::getYaw(pose.orientation);
// 短角差累加：处理±π跨界，防止绕远转向。
  yaw = last_yaw + std::atan2(std::sin(yaw - last_yaw), std::cos(yaw - last_yaw));
  mav_trajectory_generation::Vertex yaw_vertex(1);
  Eigen::VectorXd yaw_value(1);
  yaw_value << yaw;
// 最后一道门使用终点边界；中间门仅固定目标值。
  if (i + 1 == poseArray.poses.size())
    yaw_vertex.makeStartOrEnd(yaw_value, SNAP);
  else
    yaw_vertex.addConstraint(ORIENTATION, yaw);
  yaw_vertices.push_back(yaw_vertex);
  last_yaw = yaw;
}
```

A.3.1 分段时间与优化求解

```
std::vector<double> segment_times;
const double v_max = 8.0;
const double a_max = 4.0;
// 按航点与估计参数分配时间；这不是硬速度约束。
segment_times = estimateSegmentTimes(vertices, v_max, a_max);
// 本次倍率为2.0，给控制器更多跟踪时间。
for (auto& duration : segment_times) duration *= time_scale;
// N是系数数目，因此N=10对应最高九次。
const int N = 10;
mav_trajectory_generation::PolynomialOptimization<N> opt(D);
// 在固定分段时间下，按最小snap目标求位置系数。
opt.setupFromVertices(vertices, segment_times, SNAP);
opt.solveLinear();
mav_trajectory_generation::PolynomialOptimization<N> yaw_opt(1);
// yaw使用相同时间分段，保证朝向与位置同步。
yaw_opt.setupFromVertices(yaw_vertices, segment_times, SNAP);
yaw_opt.solveLinear();
mav_trajectory_generation::Segment::Vector segments;
opt.getTrajectory(&trajectory);
yaw_opt.getTrajectory(&yaw_trajectory);
// 求解完成后记录开始时刻，发布时据此计算进度。
trajectoryStartTime = ros::Time::now();
```

A.4 Part1.3

```
trajectory_msgs::MultiDOFJointTrajectoryPoint next_point;
// 使用经过时间，而不是每次都取轨迹起点。
double sampling_time = std::max(0.0, (ros::Time::now() - trajectoryStartTime).toSec());
// 超过总时长后固定为末端时刻，避免越界。
if (sampling_time > trajectory.getMaxTime())
  sampling_time = trajectory.getMaxTime();
next_point.time_from_start = ros::Duration(sampling_time);
using namespace mav_trajectory_generation::derivative_order;
// 从同一采样时刻读取位置、速度、加速度和yaw。
Eigen::Vector3d des_position = trajectory.evaluate(sampling_time, POSITION);
Eigen::Vector3d des_velocity = trajectory.evaluate(sampling_time, VELOCITY);
Eigen::Vector3d des_accel =
    trajectory.evaluate(sampling_time, ACCELERATION);
Eigen::VectorXd des_orientation =
    yaw_trajectory.evaluate(sampling_time, ORIENTATION);
geometry_msgs::Transform transform;
transform.translation.x = des_position.x();
transform.translation.y = des_position.y();
transform.translation.z = des_position.z();
// 将目标yaw转为消息所需的四元数。
transform.rotation = tf::createQuaternionMsgFromYaw(des_orientation(0));
geometry_msgs::Twist velocity, acceleration;
velocity.linear.x = des_velocity.x();
velocity.linear.y = des_velocity.y();
velocity.linear.z = des_velocity.z();
acceleration.linear.x = des_accel.x();
acceleration.linear.y = des_accel.y();
acceleration.linear.z = des_accel.z();
// 将三个消息数组填入后再发布；角速度前馈未在此设置。
next_point.transforms.push_back(transform);
next_point.velocities.push_back(velocity);
next_point.accelerations.push_back(acceleration);
desiredStatePub.publish(next_point);
```